

## ASSIGNMENT-11.1

Name-K.Bhavya Sri

H.no-2303A51863

Batch-13

### Task Description #1-Stack Implementation

**Task:** Use AI to generate a stack class with push, pop, peek, and is\_empty methods.

### Sample Input Code:

```
class Stack:
```

```
pass
```

### Expected Output:

- A functional stack implementation with all required methods and docstrings.

```
Welcome  labexamq1.py  labexamq2.py  Assignment.py X  sumofdigits.py
Assignment.py > Stack > pop
1  class Stack:
2      """
3      A Stack data structure implementation using a list.
4
5      A Stack follows the Last-In-First-Out (LIFO) principle where the last
6      element added is the first one to be removed.
7      """
8      def __init__(self):
9          """
10         Initialize an empty stack using a list.
11         """
12         self.items = []
13     def push(self, item):
14         """
15         Add an item to the top of the stack.
16
17         Args:
18             item: The item to be added to the stack.
19
20         Returns:
21             None
22         """
23         self.items.append(item)
24     def pop(self):
25         """
26         Remove and return the item from the top of the stack.
27         Returns:
28             The item at the top of the stack.
29         Raises:
30             IndexError: If the stack is empty.
31         """
32         if self.is_empty():
33             raise IndexError("pop from an empty stack")
```

```
Welcome labexamq1.py labexamq2.py Assignment.py X sumofdigits.py
Assignment.py > ...
1 class Stack:
24 def pop(self):
33     raise IndexError("pop from an empty stack")
34     return self.items.pop()
35 def peek(self):
36     """
37     Return the item at the top of the stack without removing it.
38
39     Returns:
40     |     The item at the top of the stack.
41
42     Raises:
43     |     IndexError: If the stack is empty.
44     """
45     if self.is_empty():
46         raise IndexError("peek into an empty stack")
47     return self.items[-1]
48 def is_empty(self):
49     """
50     Check if the stack is empty.
51     Returns:
52     |     True if the stack is empty, False otherwise.
53     """
54     return len(self.items) == 0
55 if __name__ == "__main__":
56     stack = Stack()
57     print(f"Is stack empty? {stack.is_empty()}")
58     stack.push(10)
59     stack.push(20)
60     stack.push(30)
61     print(f"Is stack empty after pushing items? {stack.is_empty()}")
62     print(f"Peek at top: {stack.peek()}")
63     print(f"Popped item: {stack.pop()}") # Output: 30
64     print(f"Peek at top after pop: {stack.peek()}") # Output: 20
65     print(f"Popped item: {stack.pop()}") # Output: 20
66     print(f"Popped item: {stack.pop()}") # Output: 10
67     print(f"Is stack empty? {stack.is_empty()}") # Output: True
```

PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe sers/Bhavya/OneDrive/ai coding/Assignment.py"

```
Is stack empty? True
Is stack empty after pushing items? False
Peek at top: 30
Popped item: 30
Peek at top after pop: 20
Popped item: 20
Popped item: 10
Is stack empty? True
PS C:\Users\Bhavya\OneDrive\ai coding>
```

### Explanation:

Implemented a complete stack class in the Assignment.py with all required methods:

- 1)push(item)-Adds an item to the top of the stack.
  - 2)pop()-Removes and returns the top item(raises error if empty)
  - 3)peek()-Returns the top item without removing it(raises error if empty)
  - 4)is\_empty()-Checks if the stack is empty
- Comprehensive docstrings for the class and each method
  - proper error handling with descriptive messages
  - Example usage demonstrating all methods
  - The example code follows LIFO(Last-In-First-Out)principle

### Task Description #2-Queue Implementation

**Task:**Use AI to implement a Queue using Python lists.

#### Sample Input Code:

```
class Queue:pass
```

#### Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

```
queue.py > Queue > peek
1  class Queue:
2      """A Queue data structure implementation using a list.
3      A Queue follows the First-In-First-Out (FIFO) principle where the first
4      element added is the first one to be removed."""
5      def __init__(self):
6          """ Initialize an empty queue using a list."""
7          self.items = []
8      def enqueue(self, item):
9          """
10         Add an item to the rear of the queue.
11         Args:
12             item: The item to be added to the queue.
13         Returns:
14             None
15         """
16         self.items.append(item)
17      def dequeue(self):
18          """
19         Remove and return the item from the front of the queue.
20         Returns:
21             The item at the front of the queue.
22         Raises:
23             IndexError: If the queue is empty.
24         """
```

```

25         if self.is_empty():
26             raise IndexError("dequeue from an empty queue")
27         return self.items.pop(0)
28     def peek(self):
29         """
30         Return the item at the front of the queue without removing it.
31         Returns:
32             The item at the front of the queue.
33         Raises:
34             IndexError: If the queue is empty.
35         """
36         if self.is_empty():
37             raise IndexError("peek into an empty queue")

```

Welcome labexamq1.py labexamq2.py Assignment.py queue.py X sumofdigits.py

queue.py > Queue > peek

```

1  class Queue:
28     def peek(self):
36         if self.is_empty():
37             raise IndexError("peek into an empty queue")
38         return self.items[0]
39     def size(self):
40         """
41         Return the number of items in the queue.
42         Returns:
43             The number of items currently in the queue.
44         """
45         return len(self.items)
46     def is_empty(self):
47         """
48         Check if the queue is empty.
49         Returns:
50             True if the queue is empty, False otherwise.
51         """
52         return len(self.items) == 0
53     # Example usage and testing
54     if __name__ == "__main__":
55         # Create a new queue
56         queue = Queue()
57         print("--- Queue Testing ---")
58         print(f"Is queue empty? {queue.is_empty()}") # Output: True
59         queue.enqueue(10)
60         queue.enqueue(20)
61         queue.enqueue(30)

```

```

59     queue.enqueue(10)
60     queue.enqueue(20)
61     queue.enqueue(30)
62     print(f"Is queue empty after enqueueing items? {queue.is_empty()}") # Output: False
63     print(f"Queue size: {queue.size()}") # Output: 3
64     print(f"Peek at front: {queue.peek()}") # Output: 10
65     print(f"Dequeued item: {queue.dequeue()}") # Output: 10
66     print(f"Peek at front after dequeue: {queue.peek()}") # Output: 20
67     print(f"Queue size: {queue.size()}") # Output: 2
68     print(f"Dequeued item: {queue.dequeue()}") # Output: 20
69     print(f"Dequeued item: {queue.dequeue()}") # Output: 30
70     print(f"Is queue empty? {queue.is_empty()}") # Output: True

```

```

▼ TERMINAL
● PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/Bhavya/OneDrive/ai coding/queue.py"
--- Queue Testing ---
Is queue empty? True
Is queue empty after enqueueing items? False
Queue size: 3
Peek at front: 10
Dequeued item: 10
Peek at front after dequeue: 20
Queue size: 2
Dequeued item: 20
Dequeued item: 30
Is queue empty? True
○ PS C:\Users\Bhavya\OneDrive\ai coding>

```

### Explanation:

- 1) enqueue(item)-Adds an item to the rear of the queue.
  - 2) dequeue()-Removes and returns the front item(raises error if empty).
  - 3) peek()-Returns the front item without removing it(raises error if empty).
  - 4) size()-Returns the number of items in the queue.
  - 5) is\_empty()-Checks if the queue is empty.
- Comprehensive docstrings for the class and each method.
  - Proper error handling with descriptive messages.
  - Queue follows FIFO(First-In-First-Out) principle.

### Task Description#3-LinkedList

**Task:** Use AI to generate a Singly LinkedList with insert and display methods.

#### Sample Input Code:

```
class Node:
```

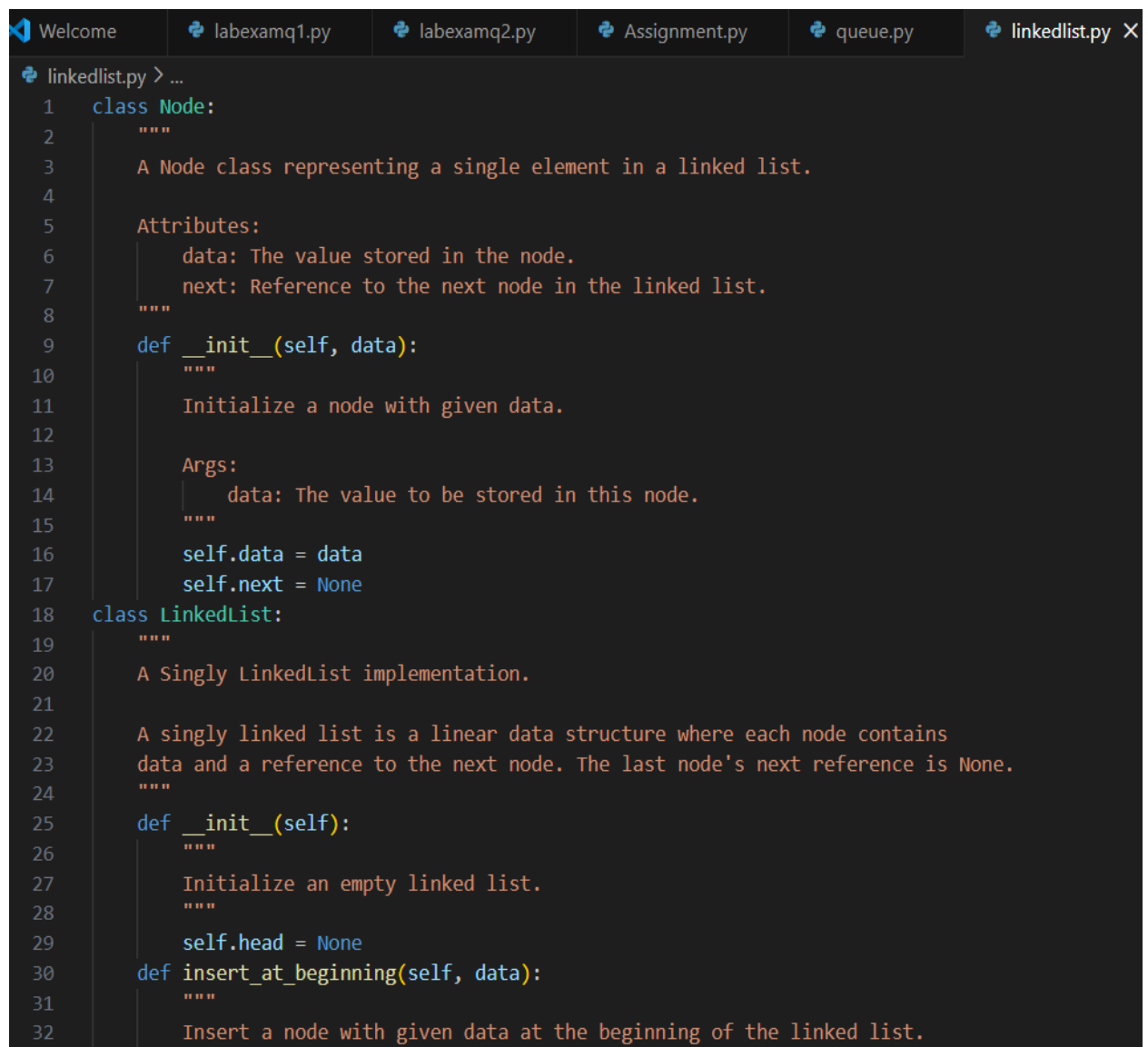
```
pass
```

```
class LinkedList:
```

```
pass
```

#### Expected Output:

- A working linked list implementation with clear method documentation.



```
Welcome  labexamq1.py  labexamq2.py  Assignment.py  queue.py  linkedlist.py X
linkedlist.py > ...
1  class Node:
2      """
3      A Node class representing a single element in a linked list.
4
5      Attributes:
6          data: The value stored in the node.
7          next: Reference to the next node in the linked list.
8      """
9      def __init__(self, data):
10         """
11         Initialize a node with given data.
12
13         Args:
14             data: The value to be stored in this node.
15         """
16         self.data = data
17         self.next = None
18     class LinkedList:
19         """
20         A Singly LinkedList implementation.
21
22         A singly linked list is a linear data structure where each node contains
23         data and a reference to the next node. The last node's next reference is None.
24         """
25         def __init__(self):
26             """
27             Initialize an empty linked list.
28             """
29             self.head = None
30         def insert_at_beginning(self, data):
31             """
32             Insert a node with given data at the beginning of the linked list.
```

```
Welcome labexamq1.py labexamq2.py Assignment.py queue.py linkedlist.py
linkedlist.py > ...
18 class LinkedList:
30     def insert_at_beginning(self, data):
32         """
33         Insert a node with given data at the beginning of the linked list.
34         Args:
35             data: The value to be inserted.
36
37         Returns:
38             None
39         """
40         new_node = Node(data)
41         new_node.next = self.head
42         self.head = new_node
43     def insert_at_end(self, data):
44         """
45         Insert a node with given data at the end of the linked list.
46
47         Args:
48             data: The value to be inserted.
49
50         Returns:
51             None
52         """
53         new_node = Node(data)
54         if self.head is None:
55             self.head = new_node
56             return
57         current = self.head
58         while current.next is not None:
59             current = current.next
60         current.next = new_node
61     def insert_at_position(self, data, position):
```

```
Welcome labexamq1.py labexamq2.py Assignment.py queue.py linkedlist.py X
linkedlist.py > ...
18 class LinkedList:
60     def insert_at_position(self, data, position):
62         """
63         Insert a node with given data at a specific position in the linked list.
64         Position is 0-indexed. Position 0 inserts at the beginning.
65         If position is greater than the list length, inserts at the end.
66         Args:
67             data: The value to be inserted.
68             position: The position at which to insert (0-indexed).
69         Returns:
70             None
71         """
72         new_node = Node(data)
73         if position == 0:
74             new_node.next = self.head
75             self.head = new_node
76             return
```

```

74         self.head = new_node
75         return
76     current = self.head
77     prev = None
78     count = 0
79     while current is not None and count < position:
80         prev = current
81         current = current.next
82         count += 1
83     if prev is not None:
84         new_node.next = current
85         prev.next = new_node
86 def display(self):
87     """Display all elements in the linked list.
88     Prints the linked list in the format: 1 -> 2 -> 3 -> None
89     Returns:
90         None"""
91     elements = []

```

Welcome labexamq1.py labexamq2.py Assignment.py queue.py linkedlist.py X

```

linkedlist.py > ...
18 class LinkedList:
86     def display(self):
91         elements = []
92         current = self.head
93         while current is not None:
94             elements.append(str(current.data))
95             current = current.next
96         print(" -> ".join(elements) + " -> None")
97     def display_as_list(self):
98         """ Return all elements in the linked list as a Python list.
99         Returns:
100             A list containing all elements in the linked list."""
101         elements = []
102         current = self.head
103         while current is not None:
104             elements.append(current.data)
105             current = current.next
106         return elements
107     def is_empty(self):
108         """
109         Check if the linked list is empty.
110         Returns:
111             True if the linked list is empty, False otherwise.
112         """
113         return self.head is None
114     def get_length(self):
115         """
116         Get the length of the linked list.
117         Returns:
118             The number of nodes in the linked list.
119         """
120         count = 0

```



```

119
120         count = 0
121         current = self.head
122         while current is not None:
123             count += 1
124             current = current.next
125         return count
126 if __name__ == "__main__":
127     ll = LinkedList()
128     print("--- Linked List Testing ---")
129     print(f"Is linked list empty? {ll.is_empty()}") # Output: True
130     print("\nInserting 10, 20, 30 at the end:")
131     ll.insert_at_end(10)
132     ll.insert_at_end(20)
133     ll.insert_at_end(30)
134     ll.display() # Output: 10 -> 20 -> 30 -> None
135     print("\nInserting 5 at the beginning:")
136     ll.insert_at_beginning(5)
137     ll.display() # Output: 5 -> 10 -> 20 -> 30 -> None
138     # Insert element at specific position
139     print("\nInserting 15 at position 2:")
140     ll.insert_at_position(15, 2)
141     ll.display() # Output: 5 -> 10 -> 15 -> 20 -> 30 -> None
142     # Check length and if empty
143     print(f"\nLinked list length: {ll.get_length()}") # Output: 5
144     print(f"Is linked list empty? {ll.is_empty()}") # Output: False
145     # Display as Python list
146     print(f"Linked list as Python list: {ll.display_as_list()}") # Output: [5, 10, 15, 20, 30]

```

```

PS C:\Users\Bhavaya\OneDrive\ai coding> & C:/Users/Bhavaya/AppData/Local/Programs/Python/Python314/python
.exe "c:/Users/Bhavaya/OneDrive/ai coding/linkedlist.py"
--- Linked List Testing ---
Is linked list empty? True

Inserting 10, 20, 30 at the end:
10 -> 20 -> 30 -> None

Inserting 5 at the beginning:
5 -> 10 -> 20 -> 30 -> None

Inserting 15 at position 2:
5 -> 10 -> 15 -> 20 -> 30 -> None

Linked list length: 5
Is linked list empty? False
Linked list as Python list: [5, 10, 15, 20, 30]
PS C:\Users\Bhavaya\OneDrive\ai coding>

```

### Explanation:

- 1)insert\_at\_beginning(data)-Inserts at the start &insert\_at\_end(data)-Inserts at the end.
  - 2)insert\_at\_position(data,position)-Insert at the specific position & display()-prints the linked list.
  - 3)display\_as\_list()-Returns elements as a python list & is\_empty()-Checks if the list is empty.
  - 4)get\_length()-Returns the number of nodes
- All methods include comprehensive docstrings.

## Task Description #4-Binary Search Tree(BST)

**Task:** Use AI to create a BST with insert and in-order traversal methods.

### Sample Input Code:

```
class BST:
```

```
    pass
```

### Expected Output:

- BST implementation with recursive insert and traversal methods.

```
Welcome  labexamq1.py  labexamq2.py  Assignment.py  queue.py  linkedlist.py  BST.py
BST.py > BST > _insert_recursive
1  class Node:
2      """A Node class representing a single element in a Binary Search Tree.
3      Attributes:
4          data: The value stored in the node.
5          left: Reference to the left child node.
6          right: Reference to the right child node."""
7      def __init__(self, data):
8          """Initialize a node with given data.
9          Args:data: The value to be stored in this node."""
10         self.data = data
11         self.left = None
12         self.right = None
13  class BST:
14      """A Binary Search Tree (BST) implementation.
15      A Binary Search Tree is a binary tree where:
16      - For each node, all values in the left subtree are less than the node's value
17      - All values in the right subtree are greater than the node's value
18      - This property holds for every node in the tree"""
19      def __init__(self):
20          """Initialize an empty Binary Search Tree."""
21          self.root = None
22      def insert(self, data):
23          """Insert a value into the Binary Search Tree.
24          If the tree is empty, the value becomes the root.
25          Otherwise, it recursively finds the correct position based on BST properties.
26          Args:data: The value to be inserted.
27          Returns:
28              None"""
29          if self.root is None:
30              self.root = Node(data)
31          else:
32              self._insert_recursive(self.root, data)
33      def _insert_recursive(self, node, data):
34          """Helper method to recursively insert a value into the BST.
35          Args:
36              node: The current node being examined.
37              data: The value to be inserted.
```

```

37         data: The value to be inserted.
38     Returns:
39         None"""
40     if data < node.data:
41         if node.left is None:
42             node.left = Node(data)
43         else:
44             self._insert_recursive(node.left, data)
45     else:
46         if node.right is None:
47             node.right = Node(data)
48         else:
49             self._insert_recursive(node.right, data)
50     def inorder_traversal(self):
51         """Perform in-order traversal of the Binary Search Tree.
52         In-order traversal visits nodes in the order: Left -> Node -> Right
53         For a BST, this produces values in ascending order.
54         Returns:
55             A list of values in in-order sequence."""
56         result = []
57         self._inorder_recursive(self.root, result)
58         return result
59     def _inorder_recursive(self, node, result):
60         """
61         Helper method to recursively perform in-order traversal.
62         Args:
63             node: The current node being visited.
64             result: The list to accumulate the traversal results.

```

```

65             node: The current node being visited.
66             result: The list to accumulate the traversal results.
67         Returns:
68             None
69         """
70         if node is not None:
71             self._inorder_recursive(node.left, result)
72             result.append(node.data)
73             self._inorder_recursive(node.right, result)
74     def preorder_traversal(self):
75         """Perform pre-order traversal of the Binary Search Tree.
76         Pre-order traversal visits nodes in the order: Node -> Left -> Right
77         Returns:
78             A list of values in pre-order sequence."""
79         result = []
80         self._preorder_recursive(self.root, result)
81         return result
82     def _preorder_recursive(self, node, result):
83         """Helper method to recursively perform pre-order traversal.
84         Args:
85             node: The current node being visited.
86             result: The list to accumulate the traversal results.
87         Returns:
88             None
89         """
90         if node is not None:
91             result.append(node.data)
92             self._preorder_recursive(node.left, result)
93             self._preorder_recursive(node.right, result)
94     def postorder_traversal(self):
95         """Perform post-order traversal of the Binary Search Tree.
96         Post-order traversal visits nodes in the order: Left -> Right -> Node

```

```

93     Perform post-order traversal of the binary search tree.
94     Post-order traversal visits nodes in the order: Left -> Right -> Node
95     Returns:
96     | A list of values in post-order sequence.
97     result = []
98     self._postorder_recursive(self.root, result)
99     return result
100 def _postorder_recursive(self, node, result):
101     """Helper method to recursively perform post-order traversal.
102     Args:
103     | node: The current node being visited.
104     | result: The list to accumulate the traversal results.
105     Returns:
106     | None
107     """
108     if node is not None:
109         self._postorder_recursive(node.left, result)
110         self._postorder_recursive(node.right, result)
111         result.append(node.data)
112 def search(self, data):
113     """Search for a value in the Binary Search Tree.
114     Args:
115     | data: The value to search for.
116     Returns:
117     | True if the value is found, False otherwise.
118     """
119     return self._search_recursive(self.root, data)
120 def _search_recursive(self, node, data):
121     """
122     Helper method to recursively search for a value in the BST.
123     Args:
124     | node: The current node being examined.
125     | data: The value to search for.

```

Welcome
labexamq1.py
labexamq2.py
Assignment.py
queue.py
linkedlist.py
BST.py
X

BST.py
BST
\_insert\_recursive

```

13 class BST:
120 def _search_recursive(self, node, data):
125     data: The value to search for.
126     Returns:
127     | True if the value is found, False otherwise.
128     """
129     if node is None:
130         return False
131     if data == node.data:
132         return True
133     elif data < node.data:
134         return self._search_recursive(node.left, data)
135     else:
136         return self._search_recursive(node.right, data)
137 def is_empty(self):
138     """Check if the Binary Search Tree is empty.
139     Returns:
140     | True if the tree is empty, False otherwise.
141     """
142     return self.root is None
143 def get_height(self):
144     """Get the height of the Binary Search Tree.
145     Height is the number of edges in the longest path from root to leaf.
146     An empty tree has height -1, and a single node has height 0.

```

```
Welcome labexamq1.py labexamq2.py Assignment.py queue.py linkedlist.py BST.py X
BST.py > BST > _insert_recursive
13 class BST:
143     def get_height(self):
144         """Get the height of the Binary Search Tree.
145         Height is the number of edges in the longest path from root to leaf.
146         An empty tree has height -1, and a single node has height 0.
147         Returns:
148         | The height of the tree.
149         """
150         return self._height_recursive(self.root)
151     def _height_recursive(self, node):
152         """Helper method to recursively calculate tree height.
153         Args:
154         | node: The current node being examined.
155         Returns:
156         | The height of the subtree rooted at this node.
157         """
158         if node is None:
159             return -1
160         return 1 + max(self._height_recursive(node.left),
161                       self._height_recursive(node.right))
162 if __name__ == "__main__":
163     bst = BST()
164     print("--- Binary Search Tree Testing ---")
165     print(f"Is BST empty? {bst.is_empty()}")
166     values = [50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 65]
167     print(f"\nInserting values: {values}")
168     for value in values:
169         bst.insert(value)
170     print(f"Is BST empty? {bst.is_empty()}")
171     print(f"\nIn-order traversal (Ascending): {bst.inorder_traversal()}")
172     print(f"Pre-order traversal: {bst.preorder_traversal()}")
173     print(f"Post-order traversal: {bst.postorder_traversal()}")
174     print(f"\nSearch for 40: {bst.search(40)}")
175     print(f"Search for 100: {bst.search(100)}")
176     print(f"\nTree height: {bst.get_height()}")
```

```
PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python
.exe "c:/Users/Bhavya/OneDrive/ai coding/BST.py"
--- Binary Search Tree Testing ---
Is BST empty? True

Inserting values: [50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 65]
Is BST empty? False

In-order traversal (Ascending): [10, 20, 25, 30, 35, 40, 50, 60, 65, 70, 80]
Pre-order traversal: [50, 30, 20, 10, 25, 40, 35, 70, 60, 65, 80]
Post-order traversal: [10, 25, 20, 35, 40, 30, 65, 60, 80, 70, 50]

Search for 40: True
Search for 100: False

Tree height: 3
PS C:\Users\Bhavya\OneDrive\ai coding>
```

### Explanation:

- 1)insert(data)-Recursively inserts values maintaining BST properties
  - 2)inorder\_traversal()-Left→Node→Right(produces sorted output)
  - 3)preorder\_traversal()-Node→Left→Right
  - 4)postorder\_traversal()-Left→Right→Node
  - 5)search(data)-Searches for a value recursively & is\_empty()-Checks if tree is empty
  - 6)search(data)-Searches for a value recursively & get\_height()-Returns the tree height
- All methods use recursive implementations with comprehensive docstrings.

### Task Description #5-Hash Table

**Task:**Use AI to implement a hash table with basic insert,search,and delete methods.

### Sample Input Code:

class HashTable: pass

### Expected Output:

- Collision handling using chaining, with well-commented methods.

```
Assignment.py  queue.py  linkedlist.py  BST.py  HashTable.py X
HashTable.py > ...
1  class HashTable:
2      """A simple Hash Table implementation using chaining for collision handling.
3      - Uses a list of buckets; each bucket is a Python list storing (key, value) pairs.
4      - Keys are hashed using Python's built-in `hash()` and the bucket index is
5      | computed as `hash(key) % capacity`.
6      """
7      def __init__(self, capacity=16):
8          """Initialize the hash table.
9          Args:
10             capacity (int): Number of buckets to start with (should be > 0)."""
11             if capacity <= 0:
12                 raise ValueError("capacity must be a positive integer")
13             self.capacity = capacity
14             self.buckets = [[] for _ in range(capacity)]
15             self.size = 0
16         def _bucket_index(self, key):
17             """Return the bucket index for a given key."""
18             return hash(key) % self.capacity
19         def insert(self, key, value):
20             """Insert or update the (key, value) pair into the hash table.
21             If the key already exists, its value is updated.
22             Collision handling: chaining (append to bucket list).
23             """
24             idx = self._bucket_index(key)
25             bucket = self.buckets[idx]
26             for i, (k, v) in enumerate(bucket):
27                 if k == key:
28                     bucket[i] = (key, value)
29             return
```

```

24         idx = self._bucket_index(key)
25         bucket = self.buckets[idx]
26         for i, (k, v) in enumerate(bucket):
27             if k == key:
28                 bucket[i] = (key, value)
29                 return
30         bucket.append((key, value))
31         self.size += 1
32     def search(self, key):
33         """Search for a key and return its value if found, otherwise return None."""
34         idx = self._bucket_index(key)
35         bucket = self.buckets[idx]
36         for k, v in bucket:
37             if k == key:

```

Assignment.py

queue.py

linkedlist.py

BST.py

Hashtable.py X

Hashtable.py > ...

```

1     class Hashtable:
32         def search(self, key):
35             bucket = self.buckets[idx]
36             for k, v in bucket:
37                 if k == key:
38                     return v
39             return None
40         def delete(self, key):
41             """Delete the key from the hash table.
42             Returns:
43                 True if the key was present and deleted, False if the key was not found.
44             """
45             idx = self._bucket_index(key)
46             bucket = self.buckets[idx]
47             for i, (k, v) in enumerate(bucket):
48                 if k == key:
49                     bucket.pop(i)
50                     self.size -= 1
51                     return True
52             return False
53         def __len__(self):
54             """Return number of elements in the table."""
55             return self.size
56         def keys(self):
57             """Yield all keys stored in the table."""
58             for bucket in self.buckets:
59                 for k, _ in bucket:
60                     yield k
61         def values(self):
62             """Yield all values stored in the table."""
63             for bucket in self.buckets:
64                 for _, v in bucket:
65                     yield v
66         def items(self):
67             """Yield all (key, value) pairs stored in the table."""
68             for bucket in self.buckets:
69                 for kv in bucket:

```

```

66     def items(self):
67         """Yield all (key, value) pairs stored in the table."""
68         for bucket in self.buckets:
69             for kv in bucket:
70                 yield kv
71 if __name__ == "__main__":
72     ht = HashTable(capacity=8)
73     ht.insert("apple", 3)
74     ht.insert("banana", 5)
75     ht.insert("orange", 2)
76     ht.insert("apple", 10) # update
77     print("Keys:", list(ht.keys()))
78     print("Values:", list(ht.values()))
79     print("Items:", list(ht.items()))
80     print("Search apple:", ht.search("apple")) # 10
81     print("Search grape:", ht.search("grape")) # None
82     print("Delete banana:", ht.delete("banana")) # True
83     print("Delete grape:", ht.delete("grape")) # False
84     print("Items after deletion:", list(ht.items()))
85     print("Length:", len(ht))
86

```

```

PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python
.exe "c:/Users/Bhavya/OneDrive/ai coding/Hashtable.py"
Keys: ['apple', 'banana', 'orange']
Values: [10, 5, 2]
Items: [('apple', 10), ('banana', 5), ('orange', 2)]
Search apple: 10
Search grape: None
Delete banana: True
Delete grape: False
Items after deletion: [('apple', 10), ('orange', 2)]
Length: 2
PS C:\Users\Bhavya\OneDrive\ai coding>

```

### Explanation:

- 1)insert(key,value):add or update a (key,value).If key exists in its bucket the value is replaced ;otherwise the pair is appended and self.size increments.uses chaining for collisions.
  - 2)search(key):returns the value for key if present,otherwise None.
  - 3)delete(key):removes the key if present,decrements self.size ,and returns True;returns False if the key wasn't found.
  - 4)search(key):returns the value for key if present,otherwise None.
  - 5)len():returns number of stored elements(self.size)
- keys(),values(),items();generators yielding stored keys,values,and (key,value)pairs respectively.



## Task Description #6-Graph Representation

**Task:**use AI to implement a graph using an adjacency list.

### Sample Input Code:

```
class Graph:
```

```
pass
```

### Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

```
graph.py Assignment.py queue.py linkedlist.py BST.py Hashtable.py graph.py X
graph.py > Graph > dfs
1 class Graph:
2     """Graph implemented using an adjacency list.
3     Vertices are keys in a dictionary mapping to a set of neighboring vertices.
4     This implementation supports undirected and directed edges via the `directed`
5     flag."""
6     def __init__(self, directed=False):
7         """Initialize an empty graph.
8         Args:
9             directed (bool): If True, edges are directed (u -> v). Otherwise edges
10            are undirected (u <-> v)."""
11        self.adj = {}
12        self.directed = directed
13    def add_vertex(self, v):
14        """Add a vertex to the graph. If the vertex already exists, does nothing."""
15        if v not in self.adj:
16            self.adj[v] = set()
17    def add_edge(self, u, v):
18        """Add an edge between u and v. Automatically adds missing vertices.
19        For undirected graphs, adds both directions. For directed graphs, adds
20        only u -> v."""
21        self.add_vertex(u)
22        self.add_vertex(v)
23        self.adj[u].add(v)
24        if not self.directed:
25            self.adj[v].add(u)
26    def remove_vertex(self, v):
27        """Remove a vertex and all associated edges. If vertex not present, does nothing."""
28        if v not in self.adj:
29            return
30        for nbrs in self.adj.values():
31            nbrs.discard(v)
32        del self.adj[v]
33    def remove_edge(self, u, v):
34        """Remove edge u -> v. For undirected graphs also removes v -> u."""
35        if u in self.adj:
36            self.adj[u].discard(v)
```

```

35         if u in self.adj:
36             self.adj[u].discard(v)
37         if not self.directed and v in self.adj:
38             self.adj[v].discard(u)
39     def neighbors(self, v):
40         """
41         Return an iterable of neighbors for vertex v. Raises KeyError if v missing.
42         """
43         return iter(self.adj[v])
44     def has_vertex(self, v):
45         """Return True if vertex exists in the graph."""
46         return v in self.adj
47     def has_edge(self, u, v):
48         """Return True if edge u -> v exists."""
49         return u in self.adj and v in self.adj[u]
50     def display(self):
51         """Print adjacency list of the graph."""
52         for v in sorted(self.adj):
53             nbrs = ', '.join(map(str, sorted(self.adj[v])))
54             print(f"{v}: {nbrs}")
55     def bfs(self, start):
56         """
57         Breadth-first search from start vertex. Returns list of visited vertices
58         in BFS order. Raises KeyError if start not in graph.
59         """
60         if start not in self.adj:
61             raise KeyError(f"Vertex {start} not in graph")
62         visited = set()
63         order = []
64         from collections import deque
65         q = deque([start])
66         visited.add(start)
67         while q:
68             u = q.popleft()
69             order.append(u)

```

```

69             u = q.popleft()
70             order.append(u)
71             for v in self.adj[u]:
72                 if v not in visited:
73                     visited.add(v)
74                     q.append(v)
75         return order
76     def dfs(self, start):
77         """Depth-first search (iterative) from start vertex. Returns list of visited
78         vertices in DFS order. Raises KeyError if start not in graph."""
79         if start not in self.adj:
80             raise KeyError(f"Vertex {start} not in graph")
81         visited = set()
82         order = []
83         stack = [start]
84         while stack:
85             u = stack.pop()
86             if u in visited:
87                 continue
88             visited.add(u)
89             order.append(u)

```

```
86         continue
87         visited.add(u)
88         order.append(u)
89         for v in sorted(self.adj[u], reverse=True):
90             if v not in visited:
91                 stack.append(v)
92         return order
93 if __name__ == "__main__":
94     g = Graph()
95     g.add_edge('A', 'B')
96     g.add_edge('A', 'C')
97     g.add_edge('B', 'D')
98     g.add_edge('C', 'E')
99     print("Adjacency list:")
100    g.display()
101    print("\nBFS from A:", g.bfs('A'))
102    print("DFS from A:", g.dfs('A'))
103
```

```
PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/Bhavya/OneDrive/ai coding/graph.py"
Adjacency list:
A: B, C
B: A, D
C: A, E
D: B
E: C

BFS from A: ['A', 'C', 'B', 'E', 'D']
DFS from A: ['A', 'B', 'D', 'C', 'E']
PS C:\Users\Bhavya\OneDrive\ai coding>
```

### Explanation:

- 1) Implements a graph using an adjacency list(supports directed or undirected graphs).
- 2) self.adj is a dict mapping each vertex→set of neighbouring vertices.
- 3) add\_edge auto-creates missing vertices; undirected graphs add both directions; bfs/dfs return visitation order lists.
- 4) Complexity: Adding/removing an edge or checking membership is  $O(1)$  average; traversals(bfs/s) are  $O(V+E)$ .
- 5) create: g=Graph()  
Adding: g.add\_edge('A','B')  
Showing: g.display()  
Traverse: g.bfs('A'), g.dfs('A')

## Task Description #7-Priority Queue

**Task:** Use AI to implement a priority queue using python's heapq module

### Sample Input Code:

```
class PriorityQueue:
```

```
pass
```

### Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority), and display methods.

```
ment.py  queue.py  linkedlist.py  BST.py  Hashtable.py  graph.py  priorityqueue.py X

priorityqueue.py > ...
1  import heapq
2  class PriorityQueue:
3      """A Priority Queue implementation using Python's heapq module.
4      Items are dequeued based on priority. By default, higher priority values
5      are dequeued first (max-heap behavior). Uses a min-heap internally with
6      negated priorities."""
7      def __init__(self):
8          """Initialize an empty priority queue."""
9          self.heap = []
10         self.counter = 0 # Tie-breaker for stable ordering when priorities are equal
11     def enqueue(self, item, priority):
12         """Add an item to the queue with a given priority.
13         Higher priority values are dequeued first. If two items have the same
14         priority, they are dequeued in the order they were added.
15         Args:
16             item: The data to be stored.
17             priority (int/float): The priority value. Higher = higher priority.
18         Returns:
19             None """
20         heapq.heappush(self.heap, (-priority, self.counter, item))
21         self.counter += 1
22     def dequeue(self):
23         """Remove and return the item with the highest priority.
24         Returns:
25             The item with the highest priority value.
26         Raises:
27             IndexError: If the queue is empty."""
28         if self.is_empty():
29             raise IndexError("dequeue from an empty priority queue")
30         _, _, item = heapq.heappop(self.heap)
31         return item
32     def peek(self):
33         """Return the item with the highest priority without removing it.
34         Returns:
35             The item with the highest priority value.
36         Raises:
37             IndexError: If the queue is empty.
```

```

ment.py  queue.py  linkedlist.py  BST.py  Hashtable.py  graph.py  priorityqueue.py X
priorityqueue.py > ...
2  class PriorityQueue:
32  def peek(self):
35      """
36      The item with the highest priority value.
37      Raises:
38      IndexError: If the queue is empty.
39      """
40      if self.is_empty():
41          raise IndexError("peek into an empty priority queue")
42      _, _, item = self.heap[0]
43      return item
44  def is_empty(self):
45      """Check if the priority queue is empty.
46      Returns:
47      True if the queue is empty, False otherwise.
48      """
49      return len(self.heap) == 0
50  def size(self):
51      """Return the number of items in the priority queue.
52      Returns:
53      The number of items in the queue."""
54      return len(self.heap)
55  def display(self):
56      """Display all items in the priority queue in priority order (highest first).
57      Prints items along with their actual (non-negated) priority values. """
58      if self.is_empty():
59          print("Priority Queue is empty")
60          return
61      items_with_priority = [(-priority, item) for priority, _, item in self.heap]
62      items_with_priority.sort(reverse=True)
63      print("Priority Queue (highest to lowest priority):")
64      for priority, item in items_with_priority:
65          print(f"  Item: {item}, Priority: {priority}")
66  if __name__ == "__main__":
67      pq = PriorityQueue()
68      print("--- Priority Queue Testing ---")
69      print(f"Is priority queue empty? {pq.is_empty()}") # True
70      print("\nEnqueueing items:")

```

```

71      print(f"  Item: {item}, Priority: {priority}")
72  if __name__ == "__main__":
73      pq = PriorityQueue()
74      print("--- Priority Queue Testing ---")
75      print(f"Is priority queue empty? {pq.is_empty()}") # True
76      print("\nEnqueueing items:")
77      pq.enqueue("Task A", 3)
78      pq.enqueue("Task B", 5)
79      pq.enqueue("Task C", 1)
80      pq.enqueue("Task D", 5)
81      pq.enqueue("Task E", 2)
82      pq.display()
83      print(f"\nQueue size: {pq.size()}") # 5
84      print(f"Peek at highest priority: {pq.peek()}") # Task B or Task D (priority 5)
85      print("\nDequeuing items in priority order:")
86      while not pq.is_empty():
87          item = pq.dequeue()
88          print(f"  Dequeued: {item}")
89
90      print(f"\nIs priority queue empty? {pq.is_empty()}") # True

```

### Explanation:

- 1) enqueue(item, priority)-adds item with priority (higher values=higher priority)
- 2) dequeue()-removes and returns the highest-priority item
- 3) peek()-returns highest-priority item without removing
- 4) display()-shows all items sorted by priority (highest first)
- 5) is\_empty(), size()-utility methods

### Task Description #8 – Deque

**Task:** Use AI to implement a double-ended queue using collections.deque.

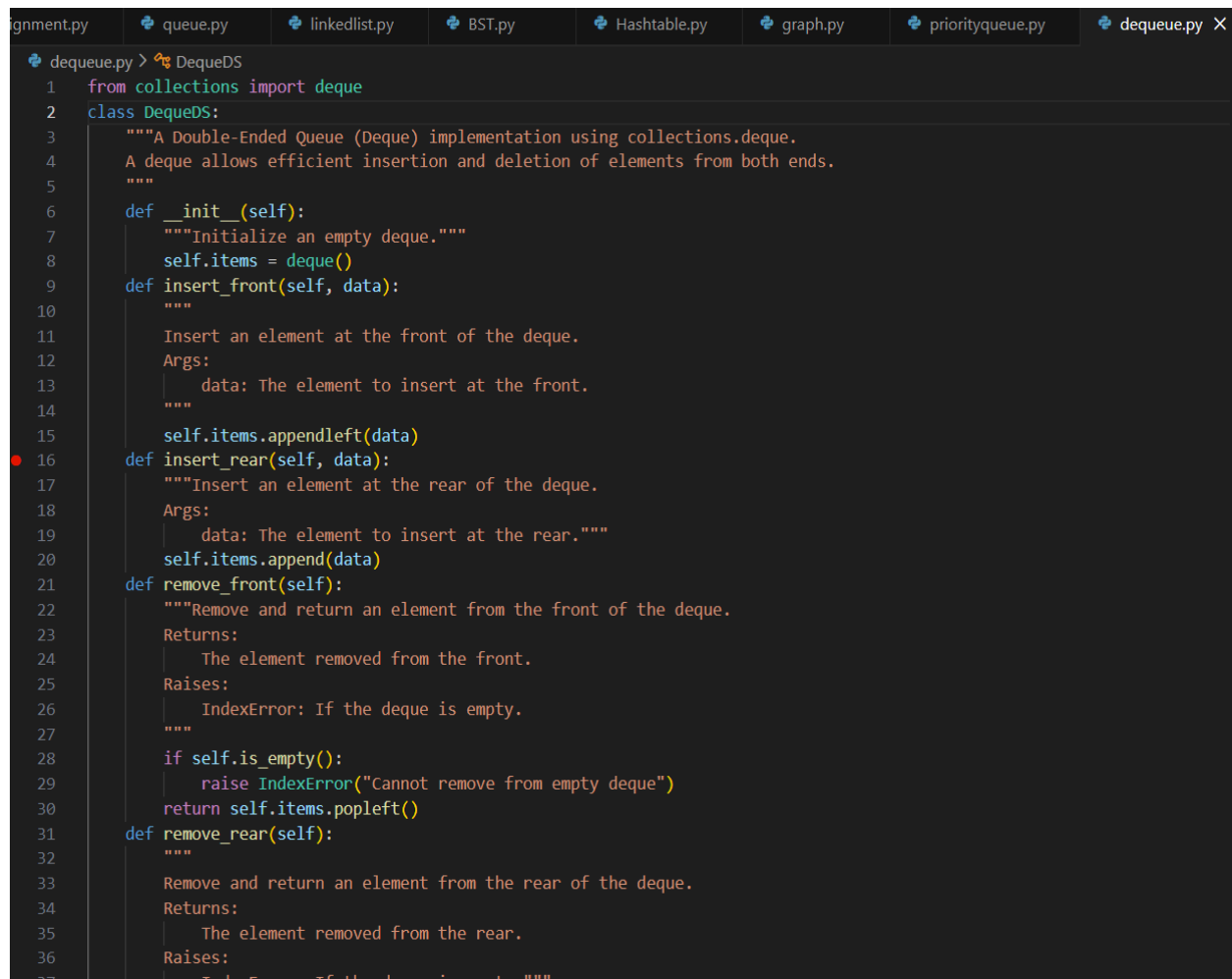
### Sample Input Code:

class DequeDS:

pass

### Expected Output:

- Insert and remove from both ends with docstrings.



```
gnment.py  queue.py  linkedlist.py  BST.py  Hashtable.py  graph.py  priorityqueue.py  dequeue.py X
dequeue.py > DequeDS
1  from collections import deque
2  class DequeDS:
3      """A Double-Ended Queue (Deque) implementation using collections.deque.
4      A deque allows efficient insertion and deletion of elements from both ends.
5      """
6      def __init__(self):
7          """Initialize an empty deque."""
8          self.items = deque()
9      def insert_front(self, data):
10         """
11         Insert an element at the front of the deque.
12         Args:
13             data: The element to insert at the front.
14         """
15         self.items.appendleft(data)
16      def insert_rear(self, data):
17         """Insert an element at the rear of the deque.
18         Args:
19             data: The element to insert at the rear."""
20         self.items.append(data)
21      def remove_front(self):
22         """Remove and return an element from the front of the deque.
23         Returns:
24             The element removed from the front.
25         Raises:
26             IndexError: If the deque is empty.
27         """
28         if self.is_empty():
29             raise IndexError("Cannot remove from empty deque")
30         return self.items.popleft()
31      def remove_rear(self):
32         """
33         Remove and return an element from the rear of the deque.
34         Returns:
35             The element removed from the rear.
36         Raises:
37             IndexError: If the deque is empty. """
```

```

36     Raises:
37         IndexError: If the deque is empty."""
38     if self.is_empty():
39         raise IndexError("Cannot remove from empty deque")
40     return self.items.pop()
41 def is_empty(self):
42     """
43     Check if the deque is empty.
44     Returns:
45         True if the deque is empty, False otherwise.
46     """
47     return len(self.items) == 0
48 def size(self):
49     """
50     Get the number of elements in the deque.
51     Returns:
52         The number of elements in the deque."""
53     return len(self.items)
54 def display(self):
55     """Display the elements of the deque.
56     Returns:
57         A string representation of the deque."""
58     return str(list(self.items))
59 if __name__ == "__main__":
60     dq = DequeDS()
61     print("Inserting elements...")
62     dq.insert_rear(10)
63     dq.insert_rear(20)
64     dq.insert_front(5)
65     dq.insert_rear(30)
66     print(f"Deque after insertions: {dq.display()}")
67     print(f"Size: {dq.size()}")

```

```

68     dq.insert_rear(30)
69     print(f"Deque after insertions: {dq.display()}")
70     print(f"Size: {dq.size()}")
71     print("\nRemoving elements...")
72     front_element = dq.remove_front()
73     print(f"Removed from front: {front_element}")
74     rear_element = dq.remove_rear()
75     print(f"Removed from rear: {rear_element}")
76     print(f"Deque after removals: {dq.display()}")
77     print(f"Size: {dq.size()}")
78     print(f"Is empty: {dq.is_empty()}")

```

✓ **TERMINAL** Python + ▾

```

PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/Bhavya/OneDrive/ai coding/dequeue.py"
Inserting elements...
Deque after insertions: [5, 10, 20, 30]
Size: 4

Removing elements...
Removed from front: 5
Removed from rear: 30
Deque after removals: [10, 20]
Size: 2
Is empty: False

```

**Explanation:**

1)insert\_front(data)-Insert element at the front(left end)

2)insert\_rear(data)-Insert element at the rear(right end)

3)remove\_front()-Remove element from the front

4)remove\_rear()-Remove element from the rear

5)is\_empty()-Check if deque is empty

6)size()-Get number of elements

7) display()-Display all elements

→All methods include comprehensive docstrings explaining their purpose,parameters,return values and exceptions.

**Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure**

**Scenario:**Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

**Student Task:**

- For each feature, select the most appropriate data structure from the list below:
  - o Stack o Queue
  - o Priority Queue
  - o Linked List
  - o Binary Search Tree (BST)
  - o Graph o Hash Table
  - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

**Expected Output:**

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.



```

queue.py linkedlist.py BST.py Hashtable.py graph.py priorityqueue.py dequeue.py application.py X
application.py > ...
1 class EventRegistration:
2     """Simple Event Registration System using Hash Table."""
3     def __init__(self):
4         """Initialize with empty events dictionary."""
5         self.events = {}
6     def create_event(self, event_id, event_name, capacity):
7         """Create a new event."""
8         self.events[event_id] = {
9             'name': event_name,
10            'capacity': capacity,
11            'participants': {}
12        }
13        print(f"✓ Event '{event_name}' created (Capacity: {capacity})")
14    def register(self, event_id, participant_id, name):
15        """Register a participant. Returns True if successful."""
16        if event_id not in self.events:
17            print(f"X Event not found")
18            return False
19        event = self.events[event_id]
20        if len(event['participants']) >= event['capacity']:
21            print(f"X Event full")
22            return False
23        if participant_id in event['participants']:
24            print(f"X Already registered")
25            return False
26        event['participants'][participant_id] = name
27        print(f"✓ {name} registered for '{event['name']}'")
28        return True
29    def remove(self, event_id, participant_id):
30        """Remove a participant."""
31        if event_id in self.events and participant_id in self.events[event_id]['participants']:
32            name = self.events[event_id]['participants'].pop(participant_id)
33            print(f"✓ {name} removed")
34            return True
35        print(f"X Participant not found")
36        return False
37    def show_participants(self, event_id):
38        """Show all participants in an event."""
39        if event_id not in self.events:
40            print(f"X Event not found")
41            return
42        event = self.events[event_id]
43        count = len(event['participants'])
44        capacity = event['capacity']
45        print(f"\n{event['name']}: {count}/{capacity} participants")
46        for pid, name in event['participants'].items():
47            print(f"    - {pid}: {name}")
48 if __name__ == "__main__":

```

```

48 if __name__ == "__main__":
49     reg = EventRegistration()
50     reg.create_event("E001", "Tech Summit", 3)
51     reg.create_event("E002", "AI Workshop", 2)
52     print("\n--- Registrations ---")
53     reg.register("E001", "P1", "Alice")
54     reg.register("E001", "P2", "Bob")
55     reg.register("E001", "P3", "Charlie")
56     reg.register("E001", "P4", "Diana") # Over capacity
57     reg.show_participants("E001")
58     print("\n--- Removal ---")
59     reg.remove("E001", "P2")
60     reg.show_participants("E001")
61

```

```

▼ TERMINAL Python + v
● PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/Bhavya/OneDrive/ai coding/a
pplication.py"
✓ Event 'Tech Summit' created (Capacity: 3)
✓ Event 'AI Workshop' created (Capacity: 2)

--- Registrations ---
✓ Alice registered for 'Tech Summit'
✓ Bob registered for 'Tech Summit'
✓ Charlie registered for 'Tech Summit'
X Event full

Tech Summit: 3/3 participants
- P1: Alice
- P2: Bob
- P3: Charlie

--- Removal ---
✓ Bob removed

Tech Summit: 2/3 participants
- P1: Alice
- P3: Charlie

```

### Explanation:

- 1) create\_event()-Create events with capacity
- 2) register()-register participants (with capacity & duplicate checks)
- 3) remove()-Remove participants
- 4) remove()-remove participants
- 5) show\_participants()-Display all registered participants

| Feature                     | Data Structure | Justification                                                                                              |
|-----------------------------|----------------|------------------------------------------------------------------------------------------------------------|
| Student Attendance Tracking | Hash Table     | $O(1)$ lookup to check student status (in/out) and quickly update their presence records                   |
| Event Registration System   | Hash Table     | $O(1)$ search and removal of participants; ideal for instant registration confirmation and easy management |
| Library Book Borrowing      | Priority Queue | Efficiently tracks and prioritizes overdue books; $O(1)$ access to most urgent items                       |
| Bus Scheduling System       | Graph          | Models stops as vertices and routes as edges; perfect for pathfinding and visualizing connections          |
| Cafeteria Order Queue       | Queue (Deque)  | Implements FIFO service; students served in arrival order with $O(1)$ enqueue/dequeue                      |

### Task Description #10: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

#### Student Task:

- For each feature, select the most appropriate data structure from the list below:
  - o Stack o Queue o Priority Queue o Linked List o Binary Search Tree (BST)
  - o Graph o Hash Table o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

#### Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Assignment.py    dequeue.py    application.py    ecommerce.py X

ecommerce.py > ...

```
1  from collections import deque
2  class ShoppingCart:
3      """Shopping cart using Deque for dynamic add/remove."""
4      def __init__(self):
5          """Initialize empty cart."""
6          self.items = deque()
7      def add_product(self, product_id, product_name, price, quantity=1):
8          """Add product to cart."""
9          self.items.append({'id': product_id, 'name': product_name, 'price': price, 'qty': quantity})
10         print(f"✓ Added {quantity} x {product_name} (${price})")
11     def remove_product(self, product_id):
12         """Remove product from cart."""
13         for item in self.items:
14             if item['id'] == product_id:
15                 self.items.remove(item)
16                 print(f"✓ Removed {item['name']}")
17                 return True
18         print(f"X Product not found")
19         return False
20     def get_total(self):
21         """Calculate total cart value."""
22         return sum(item['price'] * item['qty'] for item in self.items)
23     def display_cart(self):
24         """Show cart contents."""
25         if not self.items:
26             print("Cart is empty")
27             return
28         print("\n--- SHOPPING CART ---")
29         total = 0
30         for item in self.items:
31             subtotal = item['price'] * item['qty']
32             print(f"{item['name']} x{item['qty']}: ${subtotal:.2f}")
33             total += subtotal
34         print(f"Total: ${total:.2f}\n")
35 if __name__ == "__main__":
36     cart = ShoppingCart()
```

```
35 if __name__ == "__main__":
36     cart = ShoppingCart()
37     print("--- Adding Products ---")
38     cart.add_product(101, "Laptop", 999.99, 1)
39     cart.add_product(102, "Mouse", 29.99, 2)
40     cart.add_product(103, "Keyboard", 79.99, 1)
41     cart.display_cart()
42     print("--- Removing Product ---")
43     cart.remove_product(102)
44     cart.display_cart()
45     print(f"Final Total: ${cart.get_total():.2f}")
46
```

```

PS C:\Users\Bhavya\OneDrive\ai coding> & C:/Users/Bhavya/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/Bhavya/OneDrive/ai coding/ecommerce.py"
--- Adding Products ---
✓ Added 1 x Laptop ($999.99)
✓ Added 2 x Mouse ($29.99)
✓ Added 1 x Keyboard ($79.99)

--- SHOPPING CART ---
Laptop x1: $999.99
Mouse x2: $59.98
Keyboard x1: $79.99
Total: $1139.96

--- Removing Product ---
✓ Removed Mouse

--- SHOPPING CART ---
Laptop x1: $999.99
Keyboard x1: $79.99
Total: $1079.98

Final Total: $1079.98

```

### Explanation:

- 1) Shopping Cart → Deque (add/remove both ends)
- 2) Order Processing → Queue (FIFO order)
- 3) Top-Selling Products → Priority Queue (rank by sales)
- 4) Product Search → hash Table ( $O(1)$  lookup)
- 5) Delivery Routes → Graph (warehouse connections)
- 6) `add_product()` - Add items with quantity & `remove_product()` - Remove by product ID
- 7) `get_total()` - Calculate cart value & `display_cart()` - show formatted cart

| FEATURE                                            | DATA STRUCTURE | JUSTIFICATION       |
|----------------------------------------------------|----------------|---------------------|
| 1. Shopping Cart Management easily from both ends  | Deque          | Add/remove products |
| 2. Order Processing System orders in arrival order | Queue          | FIFO - process      |
| 3. Top-Selling Products Tracker sales count        | Priority Queue | Rank products by    |
| 4. Product Search Engine using product ID          | Hash Table     | $O(1)$ fast lookup  |
| 5. Delivery Route Planning                         | Graph          | Connect warehouses  |